AUTONOMOUS PENETRATION ENGINE · ARCHITECTURE

# Veritas

## 自动化渗透引擎 · 架构设计 v2

在**攻击图**上做状态空间搜索——节点是立足点/资产/凭据/战利品，边是 kill-chain 动作（利用漏洞→权限跃迁），目标是达成某个访问级别或拿到某份数据。能挖 HackerOne，能从 Web 打进内网横向。

融合 6 个来源 · 经取舍而非堆砌

- Cairn — 状态空间搜索哲学
- Incalmo — 高层攻击抽象
- XBOW — 协调舰队 / 创造发现·确定裁决
- ARTEX — 双图 / CAS / 串行链 / 冷图 / trace
- Strix — 沙箱执行器 / 知识包 / 报告
- BreachWeave — Observer 监督

## 01 第一性原理

**渗透 = 在「攻击图」上搜索一条从初始立足点通往目标权限/数据的路径。**每成功利用一个漏洞 = 一次访问权限的跃迁。进展用**访问级别**度量，不用"发现了几个漏洞"——这是与"漏洞扫描器/验证器"的根本分野。

### 访问级别阶梯（进展的度量）




## 02 推进循环（系统主流程）




⑤ 的新立足点触发新一轮内网 recon，回到 ① —— 直到达成 goal 或前沿耗尽

**核心原则：Creative discovers, Deterministic decides** XBOW

创造性由 LLM 负责（发现漏洞、规划路径）；"利用是否真的成功"由**确定性逻辑**判定（拿到 shell？dump 到数据？带外回连？）——LLM 永不自证。v1 的验证能力在这里复用，但降级为"动作成功判定"，服务推进而非列漏洞。

## 03 分层架构

五个处理层（人 → 协调 → 舰队 → 执行 → 攻击图）+ 一条横切能力带。右上角色块标注每层的设计血缘。

### Operator（人）

目标 / scope（含网段）/ 授权 / HITL 审批

### Coordinator 协调者（持久、唯一规划者）

ARTEX BW

读攻击图态势 → 按"访问跃迁收益"排序前沿 · 持有跨唤醒串行利用链 todolist（软状态，图为权威）· 事件驱动+debounce · Observer 旁路监督防偏移/早停

### Solver 舰队 ×N（并行，每个 fresh session）

XBOW ARTEX

CAS 领取前沿 · 默认按【分解】并行（不同资产/不同 kill-chain 步）· 单点难题才启局部【冗余重试】· 舰队间 search_all_worker_traces 共享过程与凭据（排除自己）

### Executor 执行层（混合策略）

Incalmo Strix

**A · 确定性动作执行器** Incalmo

Scan / CrackHash / Login / LateralMove / OOB 验证 — 模式固定、确定性强、省上下文、可审计

**B · 自由沙箱执行器** Strix

Web 漏洞挖掘与利用 — agent + Kali 沙箱 shell + 代理 + 浏览器 + 知识包，随机应变

两者结果都归一化写回攻击图 · "利用成功"确定性判定（creative/deterministic）

### Attack Graph 攻击图状态服务（= 黑板 / Stigmergy）

Cairn ARTEX

双图：资产真值图（全局，DB 唯一索引去重，upsert 自动串父子）+ 推进血缘图（每任务，kill-chain 边）+ 锚点溯源

### 横切能力

| | | |
|---|---|---|
| 知识层 markdown skills（preload+按需） Strix | 记录型 MITM 留痕 + 出口代理 + CA 注入 ARTEX | 容器沙箱（Kali） Strix |
| 冷图 digest 压缩 ARTEX | 分级自主 / 审批门 ARTEX | 多 provider（LiteLLM） Strix |
| 跨 run 策略记忆（向量库） 净新增 | 证据纪律 / 反蜜罐 v1 | 授权边界 scope（网段） |

## 04 攻击图数据模型

沿用 ARTEX 的双图 + 锚点，扩展出"访问权限推进"语义。资产真值图全局共享（去重交给 DB 唯一索引），推进血缘图按任务记录 kill chain。




### 关键原语

| 原语 | 含义 | 角色 |
| --- | --- | --- |
| Foothold | label + access_level + location | 状态空间的**节点**，进展度量（初始 external） |
| Opportunity | from_foothold + target_asset + kind + 期望跃迁 | 待走的一步（Coordinator 产，Solver 领） |
| Finding | 确认"这条边能走"的漏洞证据 | 不是终点，是可走的**边** |
| Credential / Loot | 凭据 / 数据战利品 | 推进与横向的**燃料**；Loot 常常本身就是 goal |

## 05 关键设计抉择（取 / 舍 / 融合）

| 抉择点 | 决定 | 理由 |
| --- | --- | --- |
| 决策/执行分层 | 混合：确定性动作走 Incalmo 式声明执行器；Web 利用走 Strix 式自由沙箱 | Web 利用千变万化，固定动作集表达不了；横向/扫描/破解/登录模式固定、是状态骨架。**按动作确定性程度分治，不强塞。** |
| 舰队策略 | 分解派生为主，局部冗余重试为辅 | 渗透是广度/多目标；全量冗余成本高。ARTEX 的"fresh session+图为权威+trace 互查"已解决累积错误。 |
| 意图领取 | 条件 UPDATE CAS，弃 SKIP LOCKED | 横向多 worker 抢同一前沿，CAS 无死锁无长事务，配崩溃重领更合适。 |
| 执行器工具 | 沙箱 shell + 薄知识包，不逐个 wrap 工具 | Strix 验证过：给 Kali 沙箱跑 CLI 比为每个扫描器写 wrapper 灵活。 |
| 验证能力 | 降级为 Executor 的"动作成功判定" | v1 没白做——变成"利用是否真成功"的判定，服务推进而非列漏洞。 |
| 上下文治理 | 冷图 digest 压缩 + 结构化安全 checkpoint | 多主机横向让图爆炸，冷图是长程省成本关键，且算法层可直接移植。 |
| 砍掉/后置 | SARIF/auto-fix、资产同步、Web UI、MCP、复测 | AppSec 修复/企业资产/运营设施，非挖洞+渗透内核，后期按需。 |

## 06 诚实的闭环判断

| 能力 | 能闭环 | 实际效果 |
| --- | --- | --- |
| Web 漏洞挖掘（挖 HackerOne 主体） | ✓ 能 | 达到活跃 bug hunter 的广度与速度，中低危有稳定产出；顶尖高危/业务逻辑仍需人。 |
| Web 利用链（dump→凭据→提权） | ✓ 能 | 单机 web 走完"发现→利用→提权"串行链（v1 缺的核心）。 |
| Web→内网横向 | 半 | 靶场/已知 N-day 能闭环；真实企业内网（EDR/分段）是全行业能力边界 → 人机协同。 |
| 后渗透/提权 | 半 | 已知提权可自动，复杂提权需人。 |

**定位：**自动化广度漏洞发现 + PoC 验证 + 已知路径横向。挖 HackerOne 这条线能产生真实价值且可闭环；全自动打穿真实域控是能力边界，设计上为 HITL 留接口，不假装全自动。

## 07 路线图（按"完成什么样"定义，非时间）

每个里程碑 = 一个可验证的能力状态。做完一个才进下一个。

### M0 · 骨架能推进 （下一步）

攻击图+Coordinator+Solver+Executor 接口齐备；mock 演示完整 kill chain（external→web_user→凭据→内网 host→shell），goal 达成即停、输出 kill chain。不接真实目标、不烧 token。**证明"推进"跑通。**

### M1 · 单机 Web 能真打

external recon + 自由沙箱 Web Solver（Kali shell+代理+浏览器+知识包）对真实 web 靶标自主发现并利用漏洞，产出真实 foothold/credential 写回攻击图。靶标 Juice Shop。

### M2 · Web 利用链闭环

走完多步串行链 SQLi→dump→拿 hash→crack→用凭据登录→提权到 web_admin，由共享 todolist 按依赖调度。证据纪律在线（反蜜罐）。

### M3 · 多主机横向闭环

从 web 立足点内网发现 + 声明式横向动作（凭据复用/ssh/smb/winrm）+ pivot 代理，攻击图长出内网主机并拿到 shell@内网host。**"从 web 到内网横向"的验收点。**

### M4 · 规模化 + 结论可信

并行 solver 舰队 + trace 协作 + Observer 监督 + 全链路 MITM 留痕 + kill chain 报告（带每一跳 PoC）。能并行多目标、结论不误报、报告可交付。

### M5 · 长程 + 工程化

冷图压缩 + 跨 run 策略记忆（向量库）+ 分级自主/审批门 + 多 provider + kill chain 可视化回放。

### 代码结构

```
veritas/
  core/       attack_graph(board) · coordinator · solver · state(access) · events
  executors/  deterministic/(scan,crack,login,lateral,oob) · freeform/(web_solver)
  tools/      sandbox_shell · http_proxy · browser · oob_canary
  recon/      external · internal
  knowledge/  skills/*.md(frontmatter) + loader
  runtime/    docker 沙箱(Kali) · report/ kill_chain+PoC · safety/ scope+审批+分级 · llm/ provider+compaction
schema.sql · dispatch.yaml · cli.py
```